

M16 Teaching Book

14 beginner-first production-engineering lessons

Standalone DE 2026 canonical rebuild - module-level QA candidate

DE16-01 - Production mindset

By the end of this lesson

Think like the person who must operate the pipeline tomorrow, not only the person who made it run once.

1. WATCH / VISUAL FIRST

[Luke Barousse - How I Would Learn to Be a Data Engineer \(2026\)](#) - 27:38 orchestration; 30:01 terminal/bash/git; 32:36 DataOps/CI-CD; 34:10 architecture

Close the visual before the independent task. The Teaching Book below is the self-contained course authority for the lesson.

2. Teacher mental model

A prototype is a meal you cooked once; production is a kitchen that must deliver safely every day even when staff, ingredients or equipment fail.

3. Teach it from zero

Production engineering adds ownership, repeatability, validation, safe configuration, observability, recovery and evidence. A pipeline is not production-ready because the happy path worked on your laptop. You need to know what happens when input is malformed, a dependency is late, a credential expires, a deployment breaks, or a downstream consumer receives stale data.

4. Do it with me

Open the ParcelPulse project. Trace input -> validation -> transformation -> curated output -> metrics. Then inspect which files prove tests, quality gate, security, observability and rollback readiness.

5. Expected proof

Name one failure that should block publish, one signal that should alert, and one artifact that proves recovery.

6. Common beginner trap

Treating logging, tests, security and rollback as polish to add after deployment.

7. Your turn - do not copy

Close this book section and complete task DE16-01 in 03_INDEPENDENT_PRACTICE. Save the attempt before opening Review.

STOP - explain aloud

State what risk this lesson controls, what evidence would reveal failure, and what recovery or escalation action belongs next. If you cannot say it without reading, revisit the explanation before continuing.

DE16-02 - Testing strategy for data engineering

By the end of this lesson

Design a layered test strategy instead of writing random tests.

1. WATCH / VISUAL FIRST

[Anuj Sval - Data Quality with Great Expectations](#) - Testing/data-quality workflow demonstration

Close the visual before the independent task. The Teaching Book below is the self-contained course authority for the lesson.

2. Teacher mental model

A car inspection checks different layers: individual parts, assembled systems, and road behavior. One test cannot prove everything.

3. Teach it from zero

Data-engineering tests should cover pure logic, data contracts, integration boundaries, reconciliation and failure behavior. Unit tests are fast and local. Contract/data tests protect schema and business rules. Integration tests prove components cooperate. End-to-end tests are fewer because they are slower and more fragile. Production checks continue after deployment because some failures only appear with real data and timing.

4. Do it with me

Run pytest on the bundled project. Read each test name and classify it as logic, data-quality, security, observability or integration evidence.

5. Expected proof

Explain why 100 passing unit tests cannot prove that a cloud identity has correct permissions.

6. Common beginner trap

Measuring test quality by test count instead of risk coverage.

7. Your turn - do not copy

Close this book section and complete task DE16-02 in 03_INDEPENDENT_PRACTICE. Save the attempt before opening Review.

STOP - explain aloud

State what risk this lesson controls, what evidence would reveal failure, and what recovery or escalation action belongs next. If you cannot say it without reading, revisit the explanation before continuing.

DE16-03 - Automated quality gates

By the end of this lesson

Build quality gates that block dangerous output but do not make harmless warnings look fatal.

1. WATCH / VISUAL FIRST

[Anuj Sval - Data Quality with Great Expectations](#) - Expectation/validation section; map to blocking vs warning gates

Close the visual before the independent task. The Teaching Book below is the self-contained course authority for the lesson.

2. Teacher mental model

Airport security has hard stop conditions and informational notices. Treating every notice as a shutdown creates chaos; ignoring hard stops creates risk.

3. Teach it from zero

A quality gate is an explicit policy between pipeline stages. Blocking checks protect correctness or safety: duplicate business keys, invalid required values, reconciliation failure, impossible amounts. Warning checks can surface anomalies that deserve attention without necessarily stopping delivery. Every gate needs an owner, evidence, severity, and a defined recovery action.

4. Do it with me

Run `quality_gate.py` against `orders_good.csv` and `orders_bad.csv`. The good file exits 0; the bad file exits non-zero and lists blocking defects.

5. Expected proof

If a source delivers 5% fewer rows than yesterday, when should that be a warning versus a block?

6. Common beginner trap

Using an arbitrary overall quality score such as 85% to decide whether production data is safe.

7. Your turn - do not copy

Close this book section and complete task DE16-03 in 03_INDEPENDENT_PRACTICE. Save the attempt before opening Review.

STOP - explain aloud

State what risk this lesson controls, what evidence would reveal failure, and what recovery or escalation action belongs next. If you cannot say it without reading, revisit the explanation before continuing.

DE16-04 - Continuous Integration fundamentals

By the end of this lesson

Explain what CI protects and what it does not.

1. WATCH / VISUAL FIRST

[DP-700 Full Course - CI/CD & Lifecycle Management](#) - 0:48:26 onward

[GitHub Actions documentation](#) - Current workflow behavior

Close the visual before the independent task. The Teaching Book below is the self-contained course authority for the lesson.

2. Teacher mental model

CI is a checkpoint at the factory gate: every change is inspected the same way before it joins the production line.

3. Teach it from zero

Continuous Integration automatically validates a change when code is pushed or proposed through a pull request. A useful CI job checks syntax, tests, quality/static rules and security conditions. CI does not prove production is healthy; it proves the candidate change satisfies the automated pre-merge checks you defined.

4. Do it with me

Open `.github/workflows/ci.yml`. Trace checkout -> Python setup -> test execution -> static policy checks. Notice top-level contents: read permission.

5. Expected proof

What important production failure could still occur after a green CI run?

6. Common beginner trap

Calling a workflow CI when it only builds and never validates anything.

7. Your turn - do not copy

Close this book section and complete task DE16-04 in 03_INDEPENDENT_PRACTICE. Save the attempt before opening Review.

STOP - explain aloud

State what risk this lesson controls, what evidence would reveal failure, and what recovery or escalation action belongs next. If you cannot say it without reading, revisit the explanation before continuing.

DE16-05 - GitHub Actions for data projects

By the end of this lesson

Read and design a GitHub Actions workflow with explicit triggers, jobs, steps and least-privilege permissions.

1. WATCH / VISUAL FIRST

[GitHub Actions documentation](#) - Use current quickstart/workflow syntax

[GitHub Actions secure use reference](#) - Least privilege and workflow security

[GITHUB_TOKEN concept](#) - Repository-scoped ephemeral token

Close the visual before the independent task. The Teaching Book below is the self-contained course authority for the lesson.

2. Teacher mental model

A workflow is a written shift plan: when to start, which workers run, what tools they need, and what each worker is allowed to touch.

3. Teach it from zero

GitHub Actions workflows are YAML files under `.github/workflows`. Triggers decide when a workflow runs. Jobs define isolated units of work. Steps run actions or shell commands. permissions controls GITHUB_TOKEN access; start with the minimum. For cloud authentication, prefer short-lived OIDC where supported instead of storing long-lived cloud credentials. Live workflow execution must be observed on GitHub; a valid YAML file alone is not a completed runtime proof.

4. Do it with me

Inspect `ci.yml` and `deploy_oidc_example.yml.disabled`. Compare permissions. The CI file needs contents: read. The deployment example additionally requests id-token: write only because OIDC token minting needs it.

5. Expected proof

Why does id-token: write not mean the workflow can modify arbitrary cloud resources by itself?

6. Common beginner trap

Giving contents: write or broad repository permissions because it is easier than understanding the workflow need.

7. Your turn - do not copy

Close this book section and complete task DE16-05 in 03_INDEPENDENT_PRACTICE. Save the attempt before opening Review.

STOP - explain aloud

State what risk this lesson controls, what evidence would reveal failure, and what recovery or escalation action belongs next. If you cannot say it without reading, revisit the explanation before continuing.

DE16-06 - Dev, test and prod environments

By the end of this lesson

Separate dev, test and prod configuration and evidence.

1. WATCH / VISUAL FIRST

[Microsoft Reactor - CI/CD in Microsoft Fabric: From Development to Frictionless Production - Phase 2](#) - 3:26-11:35 Git/Azure DevOps integration, deployment pipelines, Dev/Test/Prod architecture

Close the visual before the independent task. The Teaching Book below is the self-contained course authority for the lesson.

2. Teacher mental model

You rehearse in a practice room, test on a stage, and perform in the main venue. You do not use the same controls and audience for all three.

3. Teach it from zero

Environments should differ by configuration and protection, not by copy-pasted code branches. Dev favors fast iteration, test favors realistic verification, and prod favors controlled deployment, least privilege, monitoring and rollback. Values such as endpoints, schedules and resource sizes belong in environment configuration; secrets belong in secret stores, not committed config files.

4. Do it with me

Compare config/dev.yml, test.yml and prod.yml. The pipeline name stays stable while target, freshness SLA and approval behavior change.

5. Expected proof

Which settings belong in configuration, and which values must never be committed even to a private repository?

6. Common beginner trap

Keeping three nearly identical codebases instead of one codebase with controlled environment configuration.

7. Your turn - do not copy

Close this book section and complete task DE16-06 in 03_INDEPENDENT_PRACTICE. Save the attempt before opening Review.

STOP - explain aloud

State what risk this lesson controls, what evidence would reveal failure, and what recovery or escalation action belongs next. If you cannot say it without reading, revisit the explanation before continuing.

DE16-07 - Deployment and rollback

By the end of this lesson

Plan deployment and rollback using observable versioned evidence.

1. WATCH / VISUAL FIRST

[Microsoft Reactor - CI/CD in Microsoft Fabric: From Development to Frictionless Production - Phase 2](#) - 4:16-11:35 and 52:35-57:48
deployment flow and automation

Close the visual before the independent task. The Teaching Book below is the self-contained course authority for the lesson.

2. Teacher mental model

A parachute is not useful if you only discover after the jump that it was never packed.

3. Teach it from zero

A deployment needs a known artifact/version, pre-deploy checks, change record, post-deploy health validation and rollback criteria. Rollback should restore a known-good state, not just redeploy something older blindly. Data migrations require extra care because code rollback does not automatically undo data changes.

4. Do it with me

Inspect `release/release_manifest.json` and `rollback_plan.md`. Simulate a failed health check with `incident.py` and identify the version to restore.

5. Expected proof

Why can rolling back application code be unsafe after a destructive schema migration?

6. Common beginner trap

Calling rollback "git revert" without defining what happens to deployed artifacts, config and data.

7. Your turn - do not copy

Close this book section and complete task DE16-07 in 03_INDEPENDENT_PRACTICE. Save the attempt before opening Review.

STOP - explain aloud

State what risk this lesson controls, what evidence would reveal failure, and what recovery or escalation action belongs next. If you cannot say it without reading, revisit the explanation before continuing.

DE16-08 - Secrets and credential safety

By the end of this lesson

Keep credentials out of source code and reason about leak prevention.

1. WATCH / VISUAL FIRST

[GitHub - What is GitHub Secret Protection? | GitHub Explained](#) - Official explainer

[GitHub push protection](#) - Current leak-prevention behavior

[GitHub secret scanning](#) - Current scanning behavior

Close the visual before the independent task. The Teaching Book below is the self-contained course authority for the lesson.

2. Teacher mental model

A password written on the keychain defeats the lock.

3. Teach it from zero

Secrets should enter runtime through approved secret stores or environment injection. Repositories should contain names/references, not real values. GitHub secret scanning can detect known credential patterns, and push protection can block supported secrets before they land in a repository. If a real secret is exposed, removing the text is not enough: rotate/revoke the credential and clean history when required.

4. Do it with me

Run `secret_scan.py` on the normal project, then run the unit test that creates a temporary fake secret pattern and proves the scanner blocks it.

5. Expected proof

What must you do after a real credential was committed even if the commit is immediately deleted?

6. Common beginner trap

Assuming private repository equals safe place for plaintext secrets.

7. Your turn - do not copy

Close this book section and complete task DE16-08 in 03_INDEPENDENT_PRACTICE. Save the attempt before opening Review.

STOP - explain aloud

State what risk this lesson controls, what evidence would reveal failure, and what recovery or escalation action belongs next. If you cannot say it without reading, revisit the explanation before continuing.

DE16-09 - RBAC and least privilege

By the end of this lesson

Design least-privilege roles by action, resource and scope.

1. WATCH / VISUAL FIRST

[Microsoft Learn - Manage and right-size RBAC role assignments](#) - Least-privilege roles and scope

Close the visual before the independent task. The Teaching Book below is the self-contained course authority for the lesson.

2. Teacher mental model

A building badge should open the rooms required for your job, not every door in the company.

3. Teach it from zero

RBAC grants permissions to identities at a scope. Start from the job the identity performs, then grant only the actions and resources needed. Separate read, write, deploy and administrative duties where useful. Avoid wildcard permissions unless the use case truly requires them. Review access over time because old access often survives long after the original need.

4. Do it with me

Open security/roles.json. Compare reader, pipeline-writer and deployer. Run policy_checks.py to prove no role contains wildcard actions.

5. Expected proof

Why is a narrowly scoped deploy identity safer than using a human administrator token in CI?

6. Common beginner trap

Granting owner/admin because the pipeline failed with a permission error.

7. Your turn - do not copy

Close this book section and complete task DE16-09 in 03_INDEPENDENT_PRACTICE. Save the attempt before opening Review.

STOP - explain aloud

State what risk this lesson controls, what evidence would reveal failure, and what recovery or escalation action belongs next. If you cannot say it without reading, revisit the explanation before continuing.

DE16-10 - PII, encryption and masking

By the end of this lesson

Recognize PII handling choices: minimize, mask, encrypt, restrict and retain deliberately.

1. WATCH / VISUAL FIRST

[Microsoft Developer - Protect Sensitive Data with Azure AI Language PII Redaction](#) - 00:58 overview; 1:12 demo; 1:42 masking styles; 2:16 exclusions

Close the visual before the independent task. The Teaching Book below is the self-contained course authority for the lesson.

2. Teacher mental model

You do not photocopy a passport into every folder just because one process needed identity verification.

3. Teach it from zero

PII protection starts with minimization: collect and publish only what the consumer needs. Masking/redaction reduces exposed detail; encryption protects data at rest/in transit but does not replace access control; tokenization/pseudonymization can reduce direct identifiers; retention policies limit how long sensitive data persists. Classification should drive controls. The right technique depends on whether downstream users need the original value.

4. Do it with me

Run pipeline.py on good data. The curated output contains masked_email, not the raw email field. A test proves the original full email values do not appear in curated output.

5. Expected proof

Why does encryption not make a dataset safe for every employee to access?

6. Common beginner trap

Treating masking, hashing, encryption and deletion as interchangeable.

7. Your turn - do not copy

Close this book section and complete task DE16-10 in 03_INDEPENDENT_PRACTICE. Save the attempt before opening Review.

STOP - explain aloud

State what risk this lesson controls, what evidence would reveal failure, and what recovery or escalation action belongs next. If you cannot say it without reading, revisit the explanation before continuing.

DE16-11 - Metadata, lineage and governance

By the end of this lesson

Create useful metadata and lineage that answer "what is this, where did it come from, who owns it, and what breaks if it changes?"

1. WATCH / VISUAL FIRST

[Microsoft Learn Live - Microsoft Purview Data Governance](#) - 1:46 overview; 24:54 scanning; 40:03 lineage; 52:26 classification

[Microsoft Purview Data Map](#) - Current metadata/data-map model

Close the visual before the independent task. The Teaching Book below is the self-contained course authority for the lesson.

2. Teacher mental model

A parcel tracking label is useful because it connects identity, origin, destination and movement history.

3. Teach it from zero

Technical metadata describes schema, files, tables and jobs. Business metadata describes meaning, owners and terms. Operational metadata includes run status and freshness. Lineage connects upstream inputs, transformations and downstream outputs. Governance uses these signals to improve discovery, accountability, change impact and policy enforcement.

4. Do it with me

Open governance/lineage.json and governance/catalog.json. Trace orders_good.csv -> pipeline.py -> curated_orders.csv -> downstream analytics.

5. Expected proof

If a source column changes name, which downstream objects should lineage help you inspect first?

6. Common beginner trap

Calling a folder list "lineage" without recording transformation relationships.

7. Your turn - do not copy

Close this book section and complete task DE16-11 in 03_INDEPENDENT_PRACTICE. Save the attempt before opening Review.

STOP - explain aloud

State what risk this lesson controls, what evidence would reveal failure, and what recovery or escalation action belongs next. If you cannot say it without reading, revisit the explanation before continuing.

DE16-12 - Observability for data pipelines

By the end of this lesson

Instrument pipelines with signals that answer freshness, volume, quality, latency and failure questions.

1. WATCH / VISUAL FIRST

[DP-700 Full Course - Monitoring & Optimization](#) - 4:46:53-5:43:50

Close the visual before the independent task. The Teaching Book below is the self-contained course authority for the lesson.

2. Teacher mental model

A car dashboard does not show every engine detail; it shows the signals needed to know whether you can keep driving safely.

3. Teach it from zero

Data observability focuses on health of data and pipelines. Useful signals include run success, duration, input/output counts, freshness, rejected rows, schema changes and reconciliation. Alerts need thresholds, ownership and action; otherwise they become noise. Metrics should be connected to an operational question, not collected because dashboards look impressive.

4. Do it with me

Run observability.py for a healthy and stale reference time. Inspect metrics.json and the health decision.

5. Expected proof

Which metric would detect "pipeline succeeded but data is two days old"?

6. Common beginner trap

Monitoring only job status and missing stale or incomplete data.

7. Your turn - do not copy

Close this book section and complete task DE16-12 in 03_INDEPENDENT_PRACTICE. Save the attempt before opening Review.

STOP - explain aloud

State what risk this lesson controls, what evidence would reveal failure, and what recovery or escalation action belongs next. If you cannot say it without reading, revisit the explanation before continuing.

DE16-13 - Incidents, runbooks and root cause

By the end of this lesson

Respond to incidents with containment, evidence, recovery, validation and learning.

1. WATCH / VISUAL FIRST

[Microsoft Azure Friday - Azure SRE Agent: Less Toil, More Uptime, Maximum Innovation](#) - Incident workflow, runbook automation, diagnosis/root cause, post-deployment health

Close the visual before the independent task. The Teaching Book below is the self-contained course authority for the lesson.

2. Teacher mental model

Firefighting is not complete when the flames stop; you still inspect damage, restore service safely and learn why it started.

3. Teach it from zero

An incident process separates detection, triage, containment, recovery, validation and root-cause learning. A runbook gives repeatable steps for known incidents. Root cause is not the first error message; it is the underlying condition that explains why the system entered the failed state. Post-incident action should reduce recurrence or detection time.

4. Do it with me

Run incident.py with a failed publish health check. Review the generated incident record: symptom, owner, containment, rollback target, validation and follow-up.

5. Expected proof

Why is "reran job and it worked" not a root-cause analysis?

6. Common beginner trap

Changing multiple things during an incident without preserving evidence of the original failure.

7. Your turn - do not copy

Close this book section and complete task DE16-13 in 03_INDEPENDENT_PRACTICE. Save the attempt before opening Review.

STOP - explain aloud

State what risk this lesson controls, what evidence would reveal failure, and what recovery or escalation action belongs next. If you cannot say it without reading, revisit the explanation before continuing.

DE16-14 - Terraform and IaC foundations

By the end of this lesson

Explain Infrastructure as Code and write a small Terraform configuration without pretending unexecuted infrastructure exists.

1. WATCH / VISUAL FIRST

[HashiCorp - Terraform Basics](#) - 0:00-15:30

[HashiCorp - Install Terraform](#) - Current stable CLI - 1.16.2 at build audit

[Terraform language documentation](#) - Current HCL/configuration authority

Close the visual before the independent task. The Teaching Book below is the self-contained course authority for the lesson.

2. Teacher mental model

IaC is a reproducible construction plan: the plan is valuable, but the building is not real until the tools actually apply it.

3. Teach it from zero

Infrastructure as Code stores desired infrastructure/configuration in versioned text. Terraform uses HCL configuration, state, providers/resources/data and a plan/apply lifecycle. Safe learning separates syntax validation from real provisioning. In this module the bundled example uses the built-in terraform_data resource so the structure is understandable without cloud credentials. Real terraform fmt/init/validate/plan evidence is required later when the CLI is available.

4. Do it with me

Open terraform/main.tf, variables.tf and outputs.tf. Identify required_version, input variable validation, terraform_data resource and output. Run static_qa.py now; later run terraform fmt -check, init, validate and plan on a machine with Terraform 1.16.x.

5. Expected proof

Why does a successful terraform validate not prove the intended cloud resource can actually be created?

6. Common beginner trap

Putting cloud credentials directly into .tf files or committing a real state file with sensitive values.

7. Your turn - do not copy

Close this book section and complete task DE16-14 in 03_INDEPENDENT_PRACTICE. Save the attempt before opening Review.

STOP - explain aloud

State what risk this lesson controls, what evidence would reveal failure, and what recovery or escalation action belongs next. If you cannot say it without reading, revisit the explanation before continuing.